

Energy-Centered and QoS-Aware Services Selection for Internet of Things

Mohamed Essaid Khanouche, Yacine Amirat, *Member, IEEE*, Abdelghani Chibani, Moussa Kerkar, and Ali Yachir

Abstract—An important challenge to be addressed in the domain of Internet of Things (IoT) is the development of efficient services selection algorithms for an optimal management of both energy and Quality of Service (QoS) in the context of IoT services composition. This issue becomes crucial in the case of large-scale IoT environments composed of thousands of distributed entities. In this paper, an energy-centered and QoS-aware services selection algorithm (EQSA) is proposed for IoT services composition. The proposed selection approach consists of preselecting the services offering the QoS level required for user's satisfaction using a lexicographic optimization strategy and QoS constraints relaxation technique. In order to reduce the energy consumption of a composite service without affecting the user's satisfaction, the most suitable services among the preselected ones are then selected using the concept of relative dominance of services in the sense of Pareto. The relative dominance of a candidate service depends on its energy profile and QoS attributes, and user's preferences. The proposed algorithm has been evaluated through several simulation scenarios. The obtained results show clearly the good performances of the EQSA algorithm in terms of selection time, energy efficiency, composition lifetime, and optimality and its added value in comparison with algorithms dealing separately with QoS and energy consumption.

Note to Practitioners—This paper is motivated by services selection problem for an optimal management of energy and satisfaction of QoS requirements, in the context of IoT environments. The proposed selection algorithm aims at maximizing energy conservation to ensure a high services availability. It is based on the idea that it is always possible to make energy conservation by slightly reducing the QoS level without affecting user's satisfaction. Services selection is formulated and solved as a multi-objective optimization problem that consists to find a near-to-optimal solution: 1) minimizing the energy consumption of a composite service and 2) satisfying the user's requirements in terms of QoS of the composite service. Simulation results are provided to demonstrate the efficiency of the proposed approach in terms of energy conservation, time selection and optimality of found solutions, in the context of large-scale IoT environments.

Manuscript received November 13, 2015; accepted January 06, 2016. Date of publication April 13, 2016; date of current version June 30, 2016. This paper was recommended for publication by Associate Editor G. Fortino and Editor W. Shen upon evaluation of the reviewers' comments.

M. E. Khanouche is with LISSI Laboratory, UPEC University, Vitry-Sur-Seine 94400, France, and also with Laboratoire d'Informatique Médicale (LIMED), Faculté des Sciences Exactes, Université de Bejaia, 06000 Bejaia, Algérie (e-mail: mekhanouche@gmail.com).

Y. Amirat and A. Chibani are with LISSI Laboratory, UPEC University, Vitry-Sur-Seine 94400, France (e-mail: amirat@u-pec.fr; chibani@u-pec.fr).

M. Kerkar is with Laboratoire de Mécanique, Matériaux et Énergétique (L2ME), Faculté des Sciences Exactes, Université de Bejaia, 06000 Bejaia, Algérie (e-mail: m_kerkar@yahoo.fr).

A. Yachir is with LISSI Laboratory, UPEC University, Vitry-Sur-Seine 94400, France and Artificial Intelligence Laboratory, Military Polytechnic School (EMP), Algiers, Algeria (e-mail: a_yachir@yahoo.fr).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TASE.2016.2539240

Index Terms—Energy profile, Internet of Things (IoT), lexicographic optimization, multi-objective optimization, quality of service (QoS), services composition, services selection.

I. INTRODUCTION

INTERNET of Things (IoT) is a new paradigm based on the idea that smart objects and systems on the Internet are interconnected with each other, in a simple and transparent way, to seamlessly offer a multitude of services. The IoT vision can be seen as the natural continuity of paradigms coming from several domains, such as cloud computing, ubiquitous computing, service-oriented computing (SoC), and ambient intelligence (AmI) [1]. IoT is expected to offer advanced connectivity of devices, systems, and services that goes beyond machine-to-machine communications. It aims at creating open networks in which auto-organized or intelligent entities, physical or virtual objects will be interoperable and able to act independently depending on the context, circumstances or environments. According to the SoC-based IoT vision, IoT environments can be seen as smart environments composed of pervasively distributed things (e.g., devices, sensors, actuators, smartphones, and appliances) offering heterogeneous capabilities abstracted as software services [2]. In this context, the composition of IoT services is a programmatic approach for the rapid, low-cost and easy scalable development and deployment of applications such as Ambient Assisted Living, smart buildings, smart cars, and large-scale remote sensing.

The composition of IoT services promotes the creation of complex applications, i.e., composite services, by aggregating atomic services to provide new functionalities that none of the services could provide individually [3]. In this study, the term concrete service refers to an invocable service, whereas an abstract service, called also a class of services, defines, in an abstract manner, the functionality of a service. For each abstract service, there may exist several concrete services that have the same functionality but possibly with different quality levels. The composition process consists of creating a new service class by assembling together existing classes, in a kind of plan or actions flow, and to find the optimal bindings of these classes with their concrete services, called also candidate services [4]. This composition plan describes how candidate services interact with each other by specifying the order of services' invocation and rules for data transfer between these services. A key issue in a user-centered services composition is how to intelligently and effectively select the most relevant services that best match the user's requirements and closely meet the specified quality of service (QoS) level.

Despite the advancement of SoC technologies and the wide alliances formed by the major actors of the IoT to enhance the interoperability of IoT services and infrastructures (e.g., Allseen,¹ OneM2M,² IoTivity,³ and iPSO alliance⁴), various challenges still need to be investigated. In the context of large-scale IoT environments composed of thousands of distributed entities, services composition poses several critical issues, in particular the optimal management of both energy and QoS. In terms of QoS, services composition is still a challenging issue due to the dynamic and stochastic nature of IoT services. This dynamic nature is due to the frequent changes that may occur in IoT environments, whereas stochastic nature is due to the random occurrence of these changes. Several reasons can explain this situation: services can join or leave an application, services in current use may fail or their qualities decline due to mobility of users or devices, new services with better quality can appear during application lifecycle, and users expectations in terms of QoS can evolve with time and context of use. Hence, a service in a given application or service composition scheme might become unavailable or provides insufficient quality and, therefore, it should be replaced to ensure the continuity of the composite service. In terms of energy, the candidate services selected for composition are generally hosted in devices with limited energy storage capacity [5]. During the execution of a composition plan, these services may fail due to the intensive use of batteries leading to unreliable composite services, with no guarantee of their termination. In addition, batteries cannot be replaced or recharged easily in the case of large-scale deployments of IoT devices. Compared with computing and memory capabilities that grow rapidly with the miniaturization of devices, energy storage capabilities, in particular, batteries capacity grows annually by only 5% [6]. Moreover, services composition approaches that are proposed in the state of the art have some limitations when it is matter to find a compromise between energy saving and respecting user's QoS expectations. Therefore, an important challenge in the context of IoT services composition is the development of efficient algorithms for services selection, which are able to increase the batteries lifetime of IoT devices by reducing their energy consumption while satisfying as much as possible user's requirements in terms of QoS.

In this paper, an energy-centered and QoS-aware services selection algorithm (EQSA) is proposed for IoT services composition. This algorithm is based on the idea that it is always possible to make energy conservation by slightly reducing the QoS level without affecting user's satisfaction. It aims at maximizing energy conservation to ensure a high services availability. The present work is in the continuity of our previous works on services selection and composition [7]–[11]. The services selection approach, proposed in this paper, is carried out in two main phases: 1) *Preselection* of services offering the QoS level required for user's satisfaction and 2) selection of the most suitable services for composition using the concept

of *relative dominance* of services. In the preselection phase, the QoS-aware service selection is formulated as a multi-objective problem and solved using the lexicographic optimization method. This method consists of solving a sequence of single-objective subproblems; each subproblem dealing with one QoS attribute such as response time, availability, cost, etc. The QoS attributes are ranked in an order of importance depending on user's preferences. For each sub-problem, the proposed algorithm consists, first, of finding the candidate services providing the best quality with respect to the most important QoS attribute. A threshold value of quality is then defined from the best quality value and a tolerance factor characterizing the reduction of quality allowed by the user. The candidate services that do not satisfy the obtained threshold, are filtered out. This process is repeated for each QoS attribute in the order of importance of QoS attributes. The filtering phase allows the reduction of the search space of candidate services and consequently the selection time, which represents a great benefit in the case of large-scale IoT environments. The candidate services obtained from the preselection phase are compared using the dominance concept. An "overall" utility value reflecting the *relative dominance* of each candidate service within its class is calculated. In this study, the utility value of a service depends on its energy profile and QoS attributes, and user's preferences. The best services in terms of relative dominance are selected for the composition process.

The subsequent parts of the paper are organized as follows. Section II presents an overview on related work on services selection. Section III defines the service model, along with the QoS and energy models. Section IV presents the methodological foundations for solving services selection as multi-objective optimization problem. Section V details the EQSA algorithm for services selection and Section VI presents a theoretical time complexity analysis of this algorithm. The performances of the proposed approach are evaluated and discussed in Section VII. Finally, Section VIII is devoted to the conclusion and some research perspectives.

II. RELATED WORK

QoS-aware services selection for building composite services is known to be an NP-hard problem. It has been addressed in many studies. This section summarizes the most significant approaches by classifying them according to the used solving method.

Integer linear programming (ILP) has been applied to solve the services selection problem. In [12], two QoS-driven services selection approaches are proposed. The first one performs a local services selection for each services class without considering the global QoS constraints assigned to a composite service. The second approach, based on ILP, takes into account QoS constraints and user's preferences assigned to a composite service rather than to individual services classes. In [13], an approach for services selection is proposed using the mixed-integer linear programming method where the energy consumption is optimized as a QoS attribute. However, ILP time complexity increases exponentially with the scale of problem. In [14], the authors address the services selection problem in the case of severe QoS constraints imposed to a

¹[Online]. Available: <https://allseenalliance.org/framework>

²[Online]. Available: <http://www.onem2m.org/>

³[Online]. Available: <https://www.iotivity.org/>

⁴[Online]. Available: <http://www.ipso-alliance.org/>

composite service, i.e., in presence of limited resources which render the problem close to unfeasibility conditions (e.g., bounded execution time). The ILP method is improved by introducing: 1) a loop peeling technique to deal with complex composition structures and 2) a relaxation of QoS constraints to handle the cases where a feasible solution cannot be found. Global constraints are specified by the user at the composite service level whereas local constraints are specified by the designer at the atomic services level.

To overcome the limitations of ILP-based approaches, other solutions attempt to formulate the services selection problem as a combinatorial model or a graph model. In the combinatorial model, the services selection is formulated as a Multi-dimensional Multi-choice Knapsack Problem (MMKP) [15] where the graph model defines the selection as a Multi-Constraint Shortest Path problem (MCSP) [15]–[17]. For sequential composition structures, a heuristic is proposed in [15] to solve the MMKP problem, and an approach based on the Single-Source Shortest Paths algorithm is used to solve the multi-constrained service composition problem. Another algorithm is also proposed for more complex composition structures. Unlike classical composition approaches that limit the selection to a single candidate service, the approach proposed in [16] allows selecting, from a given services class, multiple candidate services that can be executed sequentially to improve the QoS of the composite service. The proposed approach improves the traditional Multiple-Constrained-Shortest-Path heuristics in terms of selection time by introducing the concept of *potential feasibility* of paths with respect to QoS requirements. In [17], the services composition is modeled as a path search problem in a directed acyclic graph (DAG). The Ant Colony meta-heuristic is used to find in the DAG a near-to-optimal path satisfying the global constraints.

Some selection approaches use decomposition-based optimization methods [18]–[20] where the global optimization problem is typically divided into suboptimization problems. Local QoS constraints are derived from the global QoS constraints. For each abstract service taken separately, a local selection is then used to find the services satisfying the corresponding local QoS constraints. The decomposition is carried out using the ILP technique in [18], a utility function in [19], and a genetic algorithm in [20].

In [7], [9]–[11], candidate services are ranked in terms of QoS using the simple additive weighting (SAW) technique that aggregates the QoS attributes into a single utility value [21]. The services selection is then transformed into a single objective optimization problem to find the candidate services providing the best utility value. For instance, the selection approach proposed in [11] takes into account two types of quality of service: static quality (SQoS) and dynamic quality (DQoS). The expected global quality of a concrete service at time t depends on the DQoS at time $t - 1$ and the SQoS which is constant.

Other approaches use machine learning techniques or other multi-objective optimization methods. For instance, in [22], a services selection algorithm is proposed to meet the global QoS constraints specified by the user. This algorithm uses the K-Means clustering technique to classify candidate services

into clusters representing QoS levels. Each cluster includes the services having roughly the same QoS. The resulting clusters are used to determine the utility values of candidate services and therefore to find near-to-optimal compositions, i.e., compositions that meet global QoS constraints and that have the highest QoS utility values. The utility value of a candidate service depends on 1) its QoS and 2) the number of candidate services of the cluster to which this service belongs. In [23], the services selection problem is addressed by representing the QoS values of services as discrete random variables with probability mass functions. First, a local selection is carried out by maximizing the probability of satisfying the local QoS constraints. The simulated annealing method is then applied to iteratively substitute some of the preselected services to maximize the probability of satisfying the global QoS constraints (called *QoS conformance*).

In [24]–[27], the QoS-based services selection problem is addressed using Pareto optimality. A Pareto set is the set of feasible solutions that have at least one objective optimized while keeping others unchanged [28]. In [24], a services composition algorithm using a partial selection approach, is proposed. Based on dominance relation, this approach allows reducing the search space by pruning unpromising candidate services in terms of QoS. A parallel implementation of this algorithm is also proposed to improve its efficiency and reduce further the search space. A dynamic services selection and composition approach taking into account events that may occur at runtime, is proposed in [25]. The concept of *replaceability*, introduced as a metric in the selection process, allows to reduce significantly the potential violation of constraints during plan execution resulting from QoS changes. The candidate services are filtered using Pareto dominance to improve the composition and reselection processes. In [26], a subset of compositions whose QoS is representative of the Pareto-optimal ones is used to approximate the Pareto set of compositions. The additive ϵ -error metric is used to estimate the precision of the approximation [29]. The proposed approach has a polynomial complexity. In [27], a Physical Service Model is proposed to describe the features of IoT physical services using four QoS attributes. Candidate physical services are evaluated with respect to user's requirements by aggregating individual QoS rating (utility) functions. The proposed selection algorithm is composed of three phases: pre-sorting according to user's preferences, dominance-based filtering, and final-sorting to select the best service.

The related work shows that QoS-aware service selection problem has been widely addressed in the literature. However, there are only few approaches dealing simultaneously with energy and QoS issues. These approaches are mostly focusing, in terms of applications, on data centers [30], [31], cloud computing [32]–[35], building automation [36], [37] or wireless sensor networks [38]. Other approaches dealing with services selection are mostly unaware of energy issues (see [7], [9], [12], [14]–[20], [22]–[27]) or consider the energy as a quality attribute [13], [39] or as an attribute equivalent to standard QoS attributes [10], [11]. For instance, in [13], [39], the authors propose a selection approach based on the minimization of the composite service global utility value formulated as a weighted sum of energy consumption and response time. However, no

TABLE I
CLASSIFICATION OF SERVICES SELECTION APPROACHES

Approach	Method used	Scalability	Energy-awareness
[7]	SAW	Not add.	No
[9]	SAW	Yes	No
[10]	SAW	Yes	Yes
[11]	SAW	Not add.	Yes
[12]	ILP	No	No
[13]	ILP	Yes	Yes
[14]	Extended ILP	No	No
[15]	MMKP/MCSP	No	No
[16]	MCSP	Yes	No
[17]	ACO	No	No
[18]	Decomposition	Yes	No
[19]	Decomposition	Yes	No
[20]	Decomposition	Yes	No
[22]	Mach. learning	No	No
[23]	Sim. annealing	No	No
[24]	Pareto Set	No	No
[25]	Pareto Set	Not add.	No
[26]	Pareto Set	Yes	No
[27]	Pareto Set	Yes	No
The proposed approach	Lex. Optimization & Pareto dominance	Yes	Yes

assessment of the energy impact on services availability, is provided. With the increasing large-scale use of emerging technologies such as IoT, the challenge related to energy efficiency-based services selection is still an open and crucial issue. Unlike the aforementioned studies, our work deals with energy efficiency of services separately from the other QoS attributes to take into account, on one hand, the user's requirements in terms of QoS and, on the other hand, energy consumption of IoT services. The proposed selection approach combines a QoS constraints relaxation technique and an optimization strategy using Pareto dominance to reduce the energy consumption of a composite service without affecting the user's satisfaction. In addition, to ensure a good balance among the candidate services in terms of energy consumption, the services consuming less energy are selected at each service composition. The aim is to provide high availability of services by minimizing the energy consumption, i.e., by maximizing the devices battery lifetime. In [24]–[27], QoS-based Pareto dominance is used to filter out the dominated services and obtain the set of Pareto-optimal (nondominated) compositions. However, the number of compositions increases exponentially with the number of services leading to an excessive computation time [26]. Moreover, in the aforementioned approaches, non-dominated candidate services may have a low quality with respect to some QoS attributes resulting therefore in a composite service with insufficient quality with respect to user's requirements. The approach proposed in this paper differs from those proposed in [24]–[27], since it consists of finding a near-to-optimal composition which is less time-consuming in comparison to [24]–[27], where all Pareto-optimal compositions are determined. In addition, in order to guarantee a composite service meeting the user's requirements in terms of QoS, candidate services that do not satisfy the QoS threshold values, are filtered out.

Table I summarizes the aforementioned approaches and classify them in terms of resolution method, scalability and energy awareness.

III. MODELS DEFINITION

A. Service Model Description

1) *Concrete Versus Abstract Service*: A concrete service cs_j is described by i) its functional properties under the form of a transformation function $Action(cs_j)$ that takes an input data vector $InputData(cs_j)$ to produce an output data vector $OutputData(cs_j)$, and ii) its non-functional properties defined by the QoS attributes vector $QoS(cs_j)$ and the energy profile $EProf(cs_j)$. Formally

$$cs_j = \{InputData(cs_j), OutputData(cs_j), Action(cs_j), QoS(cs_j), EProf(cs_j)\}. \quad (1)$$

An abstract service $AS_i = \{cs_1^i, \dots, cs_n^i\}$ is a class of n concrete services having similar functional properties, i.e., at least the same input data vector and output data vector, but that differ by their nonfunctional properties.

2) *Abstract Versus Concrete Composite Service*: An abstract composite service, described by an abstract plan, can be seen as an aggregation of abstract services (services classes) without referring to any concrete service. A concrete composite service is an instance of an abstract composite service obtained by mapping each abstract service (e.g., localization of an object) to one of its concrete services (e.g., RFID-based localization, vision-based localization).

B. Energy Profile Model

The energy profile $EProf(cs_j)$ of a concrete service cs_j is described by the following parameters: 1) the energy cost $ECost(cs_j)$ that reflects the overall energy consumption during the execution of the service, and 2) the autonomy of the service $AutoS(cs_j)$ that represents the energy level of the device hosting the service. It is estimated as follows:

$$AutoS(cs_j) = EnergyDevice(cs_j) - EnergyThreshold(cs_j) \quad (2)$$

where $EnergyDevice(cs_j)$ and $EnergyThreshold(cs_j)$ represent, respectively, the energy level of the battery-powered device hosting the service and the energy threshold value under which the device cannot provide the service anymore.

In the case of SoC-based applications, the instantaneous energy consumption of a concrete service cs_j is assumed to be constant since this service, running on the same platform, uses the same resources (networking interface, CPU occupancy, etc.) and takes the same input data vector $InputData(cs_j)$ to produce the same output data vector $OutputData(cs_j)$. The energy cost of a service cs_j can then be expressed as follows:

$$ECost(cs_j) = ConsRate(cs_j) \cdot ExecTime(cs_j) \quad (3)$$

where $ConsRate(cs_j)$ and $ExecTime(cs_j)$ represent, respectively, the consumption rate and the execution time of the service cs_j .

Finally, the energy profile of a service cs_j is defined as the ratio between its energy consumption and its autonomy as

$$EProf(cs_j) = \frac{ECost(cs_j)}{AutoS(cs_j)}. \quad (4)$$

C. QoS Model Description

1) *QoS of an Atomic Service*: QoS attributes of an atomic service can be divided into quantitative and qualitative attributes. For instance, security, privacy and comfort are examples of qualitative attributes. The quantitative QoS attributes that are most commonly used for assessing the quality of a service (see [12], [14], and [40]–[42]), are: cost, response time, reputation, reliability, and availability. The cost corresponds to service cost on both the provider and consumer sides, the response time indicates the time required by a service to fulfill a request, the reliability is the probability that a service is executed successfully, and availability reflects the probability that a service is available at the time of invocation.

Let $QoS(cs_j^i) = (qos_{1,j}^i, \dots, qos_{q,j}^i, \dots, qos_{k,j}^i)$ be the QoS attributes vector of service cs_j^i where $qos_{q,j}^i$ represents the value of the q^{th} attribute. A QoS attribute can be classified into two categories: 1) a positive attribute (e.g., availability, reliability) has a positive influence on the QoS, i.e., the more the attribute value increases, the more the QoS increases and 2) a negative attribute (e.g., response time, cost) has a negative impact on the QoS, i.e., the more the attribute value increases, the more the QoS decreases. Thus, in the context of services composition/selection, the positive attributes of a composite service have to be maximized whereas its negative attributes need to be minimized.

2) *QoS of a Composite Service*: The QoS of a composite service depends on its structure; i.e., the composition plan and the function used for aggregating QoS attributes. In terms of structure, atomic services can be connected using different types of structures: sequential, parallel, conditional, and loop. More complex structures can be designed by combining the four previous basic structures. Different operations can be used to aggregate the QoS attributes of a composite service: 1) summation (e.g., cost); 2) product (e.g., availability, reliability); and 3) min/max operator (e.g., execution time for parallel structures).

IV. SERVICES SELECTION AS MULTI-OBJECTIVE OPTIMIZATION PROBLEM

Services selection consists of binding each requested abstract service to one and only one concrete service. In this study, such a process is formulated as a multi-objective optimization problem to find a near-to-optimal solution.

- Minimizing the energy consumption of the composite service.
- Satisfying the user's requirements in terms of the QoS of the composite service.

The following definitions, given in [43], assume a minimization problem.

A. Multi-Objective Optimization Problem

A multi-objective optimization problem (MOP) that consists of minimizing n ($n \geq 2$) objectives $f_i(x)$, can be formulated as follows [43]:

$$\text{MOP} = \begin{cases} \min F(x) = (f_1(x), f_2(x), \dots, f_n(x)) \\ \text{s.t. } x \in F \end{cases} \quad (5)$$

where $x = (x_1, \dots, x_k)$ is the decision variables vector, and F represents the set of feasible solutions associated with equality and inequality constraints and explicit bounds. $F(x) = (f_1(x), \dots, f_n(x))$ is the vector of objectives to be minimized.

In practice, when objectives are in conflict, there does not exist a solution x^* that simultaneously minimizes each objective, i.e.,

$$\forall x \in F, f_i(x^*) \leq f_i(x), \quad i = 1, \dots, n. \quad (6)$$

Consequently, the *Pareto dominance* which imposes a partial order relation, can be used as a quality metric to characterize optimality.

B. Pareto Dominance

An objective vector $q = (q_1, \dots, q_n)$ dominates an objective vector $q' = (q'_1, \dots, q'_n)$, symbolically represented by $q \prec q'$, if and only if no component of q' is lower than the corresponding component of q , and at least one component of q is strictly lower than the corresponding component of q' [43]. Formally

$$\forall i \in \{1, \dots, n\} : q_i \leq q'_i \wedge \exists i \in \{1, \dots, n\} : q_i < q'_i. \quad (7)$$

Pareto optimality comes directly from the concept of dominance. A Pareto optimal solution is the feasible solution x^* that is non-dominated by any other feasible solution, i.e., there exists no feasible solution y^* such that $y^* \prec x^*$.

C. Lexicographic Optimization

The lexicographic optimization problem consists of solving a sequence of single-objective optimization subproblems ranked in the order of importance of the objectives [44]. Let $f_1, f_2, \dots, f_n$ be the sequence of objectives to be minimized, where f_1 is the primary objective and f_2 be the secondary objective, etc. The first subproblem is formulated as follows:

$$\min f_1(x), \text{ s.t. } x \in F. \quad (8)$$

If the subproblem has a unique solution, the search process provides the optimal solution x_1^* and stops. Otherwise, the subproblem associated with the secondary objective f_2 is solved under the constraint that the objective f_1 preserves its optimal value. The second subproblem is then formulated as follows:

$$\min f_2(x), \text{ s.t. } x \in F, \text{ with } f_1(x) = f_1(x_1^*). \quad (9)$$

The same process is repeated for each subproblem. The n^{th} subproblem is formulated as follows:

$$\min f_n(x), \text{ s.t. } x \in F, \text{ with } f_{n-1}(x) = f_{n-1}(x_{n-1}^*). \quad (10)$$

For a given subproblem, a relaxation may be applied to the equality constraint associated to the previous objective in the

case where a solution cannot be found. For instance, a slight decrease d_i in the optimal value of the $(i - 1)^{th}$ objective can be tolerated to obtain a tradeoff solution [43], i.e., rather than using an equality constraint, a bound constraint (threshold) is associated to the previous optimized objective. Formally

$$\min f_i(x), s.t. x \in F, \text{ with } f_{i-1}(x) \leq f_{i-1}(x_{i-1}^*) + d_i. \quad (11)$$

V. EQSA: ENERGY-CENTERED AND QOS-AWARE SERVICES SELECTION ALGORITHM

The proposed algorithm for energy-centered and QoS-aware services selection includes two phases. The first phase consists of *preselecting* the services offering the QoS level required to satisfy the user's requirements. Indeed, the probability of meeting the user's requirements increases with the QoS of the selected services. In the second phase, the *relative dominance* relation is used to select among the preselected services the best candidate services for the composition process.

Since the energy is a critical resource in the context of IoT environments, the idea behind the proposed algorithm is to make energy conservation by slightly reducing the QoS level without affecting user's satisfaction. The proposed approach has two major objectives: 1) energy conservation in order to ensure high services availability and 2) satisfying the user's requirements in terms of QoS.

A. QoS Level-Based Services Preselection

The QoS-aware services selection problem is modeled as a lexicographic optimization problem where each sub-problem deals with one QoS attribute. The first phase steps of the EQSA algorithm are as follows:

- 1) Ranking the QoS attributes in an order of importance depending on the user's preferences. The rank 1 is assigned to the QoS attribute having the highest priority.
- 2) For each abstract service $AS_i = \{cs_1^i, \dots, cs_n^i\}$ with n candidate services, and for each QoS attribute processed according to the previous ranking:
 - a) Find the candidate service cs_j^i providing the best quality value with respect to the most important QoS attribute q . Let qos_q^* be the best quality value of the QoS attribute having the highest rank.
 - b) A *threshold* value of quality $qos_q^{Threshold}$ is then calculated from the best quality value qos_q^* and a *tolerance factor* δ_q characterizing the decrease of quality $qos_q^{Decrease}$ allowed by the user. This threshold value determines the QoS level required for user's satisfaction. It is calculated as follows:

$$qos_q^{Threshold} = \begin{cases} qos_q^* - qos_q^{Decrease}, & \text{if } q \text{ is positive} \\ qos_q^* + qos_q^{Decrease} & \text{otherwise} \end{cases} \quad (12)$$

with

$$qos_q^{Decrease} = qos_q^* \cdot \delta_q \quad (13)$$

where qos_q^* , δ_q , and $qos_q^{Decrease}$ represent, respectively, the best quality value, the tolerance factor, and the decrease of quality with respect to the QoS attribute q . The value of δ_q is in the range $[0, 1]$.

- c) The candidate services that do not satisfy the threshold value $qos_q^{Threshold}$, are removed from the candidate services set of AS_i . This allows reducing the search space and implicitly the selection time.

Formally, the services selection problem (SSP_q) solved at step $q = 1, \dots, k$ is formulated as follows:

$$SSP_q : \begin{cases} qos_q^* = \text{optimal}(qos_{q,j}^i) \\ cs_j^i \in AS_i \\ s.t. qos_{s,j}^i \leq qos_s^{Threshold} & \text{for a negative attribute} \\ qos_{s,j}^i \geq qos_s^{Threshold} & \text{otherwise} \end{cases} \quad (14)$$

with $s = 1, \dots, q - 1$.

For each abstract service AS_i , the first phase of the EQSA algorithm produces a set of candidate services satisfying the required QoS level in terms of user's satisfaction (see algorithm 1). This set, called *Preselected Candidates Set* $PCS(AS_i)$, is defined as follows : $PCS(AS_i) = \{cs_j^i / qos_{q,j}^i \geq qos_q^{Threshold} \text{ if } q \text{ is a positive attribute and } qos_{q,j}^i \leq qos_q^{Threshold} \text{ otherwise, } 1 \leq q \leq k\}$.

Algorithm 1 – QoS level-based services preselection

Inputs: Abstract service AS_i with n candidate services.

δ_q : the tolerance factor for each QoS attribute q .

1: Ranking the QoS attributes according to the user's preferences. The rank is $1 \dots k$. The attribute $q = 1$ has the highest priority.

2: $PCS = AS_i$.

3: **for** $q = 1$ until k **do**

4: Find cs_j^i from PCS such as $qos_q^* = \text{optimal}(qos_{q,j}^i)$.

5: Calculate $qos_q^{Decrease}$ using (13).

6: Calculate $qos_q^{Threshold}$ using (12).

7: Update PCS as follows:

8: **if** q is a positive attribute **then**

9: $PCS = \{cs_x^i \in PCS \cap AS_i / qos_{q,x}^i \geq qos_q^{Threshold}\}$

10: **else**

11: $PCS = \{cs_x^i \in PCS \cap AS_i / qos_{q,x}^i \leq qos_q^{Threshold}\}$

12: **end if**

13: **end for**

Outputs: The Preselected Candidates Set (PCS).

B. Relative Dominance-Based Services Selection

In this phase, for each abstract service, the preselected candidate services are compared using a utility value reflecting their *relative dominance*. The relative dominance of a service depends on its energy profile and QoS attributes, and user's preferences. The best services in terms of relative dominance are selected for the composition process.

Let $cs_{i_1}^i$ and $cs_{i_2}^i$ be two candidate services belonging to $PCS(AS_i)$ and having as QoS values vectors $QoS(cs_{i_1}^i) = (qos_{1,i_1}^i, \dots, qos_{q,i_1}^i, \dots, qos_{k,i_1}^i)$ and $QoS(cs_{i_2}^i) = (qos_{1,i_2}^i, \dots, qos_{q,i_2}^i, \dots, qos_{k,i_2}^i)$, respectively.

1) *QoS-Based Pareto Dominance*: The QoS-based Pareto dominance is defined as follows: the candidate service $cs_{i_1}^i$ dominates the candidate service $cs_{i_2}^i$, represented by $cs_{i_1}^i \succ cs_{i_2}^i$, if the two conditions hold.

- $cs_{i_1}^i$ is *at least as good as* $cs_{i_2}^i$ for all the QoS attributes, i.e., $\forall q \in \{1, \dots, |QoS|\}, qos_{q,i_1}^i \geq qos_{q,i_2}^i$.
- $cs_{i_1}^i$ is *strictly better than* $cs_{i_2}^i$ for at least one QoS attribute, i.e., $\exists q \in \{1, \dots, |QoS|\}, qos_{q,i_1}^i > qos_{q,i_2}^i$.

The symbol $\geq$ means: "as good as or better than". The comparison between QoS attributes values qos_{q,i_1}^i and qos_{q,i_2}^i depends on the category of the QoS attribute q ; i.e., negative or positive. The quality of the candidate $cs_{i_1}^i$ is as good as or better than that of $cs_{i_2}^i$, i.e., $QoS(cs_{i_1}^i) \geq QoS(cs_{i_2}^i)$, if all the QoS attributes values of $cs_{i_1}^i$ are equal to or greater than those of $cs_{i_2}^i$. Formally, $QoS(cs_{i_1}^i) \geq QoS(cs_{i_2}^i)$ if $\forall q \in \{1, \dots, |QoS|\}, qos_{q,i_1}^i \geq qos_{q,i_2}^i$.

The dominance is a partial order relation and there is a case encountered in practice where the service $cs_{i_1}^i$ dominates the service $cs_{i_2}^i$ with respect to some QoS attributes while the service $cs_{i_2}^i$ dominates $cs_{i_1}^i$ for the rest of the attributes. In this case, neither $cs_{i_1}^i$ nor $cs_{i_2}^i$ dominates each other; these two services are called *nondominated* or *Pareto equivalents*. In the following example, cs_1^i , cs_2^i and cs_3^i are three candidate services, such as

$$QoS(cs_1^i) = (RT = 13, AV = 0.95)$$

$$QoS(cs_2^i) = (RT = 15, AV = 0.97)$$

$$QoS(cs_3^i) = (RT = 13, AV = 0.92)$$

where RT represents the service *response time* which is a negative QoS attribute and AV is the service *availability* which is a positive attribute. As $RT(cs_1^i) \geq RT(cs_3^i)$ and $AV(cs_1^i) > AV(cs_3^i)$, then $cs_1^i \succ cs_3^i$. However, as $RT(cs_1^i) \geq RT(cs_2^i)$ and $AV(cs_2^i) > AV(cs_1^i)$, then $cs_1^i \not\succ cs_2^i$; i.e., cs_1^i does not dominate cs_2^i . In conclusion, it is difficult to establish a strict order between cs_1^i and cs_2^i . Hence, cs_1^i and cs_2^i are Pareto equivalent services in terms of QoS.

Some approaches proposed in the literature (e.g., [24], [25]) use Pareto optimality in terms of QoS, coming directly from the dominance concept. The aim is to reduce the search space by pruning dominated candidate services without having the possibility to establish a strict order between the non-dominated candidate services; i.e., QoS equivalent services. To overcome the difficulty to compare the candidate services that can be Pareto

equivalents in terms of QoS, a total order relation between the *preselected* candidates, called *relative dominance*, is proposed in this study.

2) *Relative Dominance*: Let $EProf(cs_{i_1}^i) = ECost(cs_{i_1}^i)/AutS(cs_{i_1}^i)$ and $EProf(cs_{i_2}^i) = ECost(cs_{i_2}^i)/AutS(cs_{i_2}^i)$ be the energy profile of $cs_{i_1}^i$ and $cs_{i_2}^i$ respectively.

Definition 1: QoS Attribute-Based Dominance: The service $cs_{i_1}^i$ dominates the service $cs_{i_2}^i$ with respect to the q th QoS attribute if one of the two following conditions is satisfied.

- $qos_{q,i_1}^i > qos_{q,i_2}^i$.
- $qos_{q,i_1}^i = qos_{q,i_2}^i$ and $i_1 < i_2$.

The dominance of a candidate service cs_j^i with respect to the q^{th} QoS attribute, noted $Dom(cs_j^i, q)$, corresponds to the number of candidate services in $PCS(AS_i)$ dominated by the service cs_j^i . This dominance is a scalar value between 1 and $|PCS(AS_i)|$. The candidate service having the highest quality value with respect to the q th QoS attribute will be assigned the highest dominance which is equal to $|PCS(AS_i)|$ whereas the candidate service having the lowest quality value, will be assigned the lowest dominance which is equal to 1.

Definition 2: Energy Profile-Based Dominance: The service $cs_{i_1}^i$ dominates the service $cs_{i_2}^i$ with respect to energy profile if one of the two following conditions is satisfied.

- $EProf(cs_{i_1}^i) < EProf(cs_{i_2}^i)$.
- $EProf(cs_{i_1}^i) = EProf(cs_{i_2}^i)$ and $i_1 < i_2$.

The energy profile-based dominance of a candidate service cs_j^i , noted $EP - Dom(cs_j^i)$, is the number of candidate services belonging to $PCS(AS_i)$ that are dominated by the service cs_j^i . The candidate service providing the lowest energy profile value will have the highest energy profile-based dominance which is equal to $|PCS(AS_i)|$. Conversely, the service candidate having the highest energy profile value will have the lowest energy profile-based dominance which is equal to 1.

Definition 3: Relative Dominance of an Atomic Service: The relative dominance of a candidate service cs_j^i , noted $RelativeDom(cs_j^i)$, is a utility value calculated from its energy profile-based dominance and QoS-based dominance, and user's preferences $Pref$ with respect to each QoS attribute. Formally

$$RelativeDom(cs_j^i) = EP - Dom(cs_j^i) \cdot \sum_{q=1}^{|QoS|} Dom(cs_j^i, q) \cdot Pref_q \quad (15)$$

where the term $\sum_{q=1}^{|QoS|} Dom(cs_j^i, q) \cdot Pref_q$ represents the QoS-based dominance of the service cs_j^i .

In phase 2 of the EQSA algorithm, the candidate service having the highest relative dominance value is selected for the composition process (see algorithm 2). As the services selection is formulated as an energy consumption minimization problem, the energy profile-based dominance is used as a multiplicative term in (15) to calculate the relative dominance of a service. This allows selecting the services having a lower energy profile. Moreover, the choice of QoS-based dominance as selection criterion, instead of QoS attributes values, comes

from the Pareto dominance definition. From the example given in Section V-B1, the services cs_1^i and cs_2^i with the following QoS vectors: $QoS(cs_1^i) = (RT = 13, AV = 0.95)$ and $QoS(cs_2^i) = (RT = 15, AV = 0.97)$ are Pareto equivalents in terms of QoS. However, the best service is the one preferred by the user and/or that having the best energy profile-based dominance.

Algorithm 2 – Relative dominance-based services selection

Inputs: The PCS of abstract service AS_i

The set of user's preferences.

```

1: for each  $cs_j^i \in PCS$  do
2:   for each QoS attribute  $q$  do
3:     Determine the  $Dom(cs_j^i, q)$ .
4:   end for
5:   Determine the  $EP - Dom(cs_j^i)$ .
6:   Calculate relative dominance of  $cs_j^i$  using (15).
7: end for
8: Find the  $cs_x^i$  having the maximal relative dominance value.
```

Outputs: The selected candidate service cs_x^i .

To take into account the actual autonomy of services in terms of energy, and ensure their availability, the services selection is performed dynamically. Indeed, after selection and invocation of a service cs_j at time t , its autonomy is updated for the next selection step at time $t + 1$, as follows:

$$AutoS_{t+1}(cs_j) = AutoS_t(cs_j) - ECost_t(cs_j). \quad (16)$$

If a service cs_j is selected at time t , its energy profile increases making the service less efficient, and the probability of being invoked at time $t + 1$ decreases accordingly. The proposed selection approach ensures dynamically a good balance among the candidate services in terms of energy consumption, and guarantees a high availability of services.

VI. COMPLEXITY ANALYSIS

This section presents a time complexity analysis of the proposed services selection algorithm. This complexity depends on three factors: the number of services classes m forming an abstract composite service, the number of candidate services n per class, which is assumed to be the same for all classes, and the number of QoS attributes k .

A. Complexity of Phase 1

The QoS attributes ranking according to the user's preferences has a logarithmic time complexity of $O(k \log k)$. For a given services class, the time complexity for finding the service providing the best quality value with respect to a given QoS attribute is $O(n)$. The same time complexity is attained for the

filtering of candidate services not satisfying the QoS threshold values. Time complexity of phase 1 is then $O(k \log k) + O(k \cdot m \cdot n)$. As the number of QoS attributes is usually very small compared to n and m , this complexity becomes $O(n \cdot m)$.

B. Complexity of Phase 2

Using the *merge sort* algorithm, the dominance-based services ranking has a time complexity of $O(k \cdot p \log p)$ where p is the number of preselected candidate services. The same analysis can be made for the calculation of energy profile-based dominance which has a time complexity of $O(p \log p)$. In addition, the time complexity for finding the candidate service maximizing the relative dominance is of $O(p)$. For an abstract composite service with m services classes, time complexity of phase 2 is then $O(k \cdot m \cdot p \log p)$. As $k \ll m$, this complexity becomes $O(m \cdot p \log p)$.

Finally, the overall time complexity of the proposed algorithm is $O(n \cdot m) + O(m \cdot p \log p)$.

C. Complexity Comparison

Regarding the filtering phase, the services selection algorithms using QoS-based Pareto dominance (e.g., [24], [25]), have a complexity time of $O(k \cdot m \cdot n^2)$ [45] which is much higher than that of the proposed algorithm which has a time complexity of $O(n \cdot m)$. Compared to the aforementioned approaches, the proposed selection approach reduces drastically the time cost since its overall time complexity remains lower than $O(k \cdot m \cdot n^2)$.

VII. PERFORMANCE STUDY

Here, the performances of the EQSA algorithm are evaluated in simulation according to different scenarios. These scenarios are part of the research activities of the A2NEts European project,⁵ where the objective was to demonstrate, how scenarios of IoT services composition, can be instantiated and executed with a minimum energy footprint. These scenarios deal with the monitoring and smart metering of buildings. In addition to mobile terminals, these buildings are typically composed of hundreds or thousands of heterogeneous components such as wireless sensors based on Bluetooth LE, Zigbee, EnOcean,⁶ 6LoWPan [46], or Z-Wave [47] networking technologies and smart systems such as smart power outlets, shatters, elevators, access controls, air conditioning and surveillance cameras. Sensing and actuation functionalities of these devices are abstracted as services that are accessible for instance through REST [48] or COAP [49] protocols in IPV6 networks as well as building automation networks such as the BACnet.⁷ Concrete services are registered in IoT directories that contain the binding parameters and the corresponding services classes.

A. Simulation Environment and Methodology

The EQSA algorithm has been tested on a Java Virtual Machine, JRE 1.6, for Windows 64-bit system running on an Intel Core i7-4712HQ CPU with frequency of 2.3 GHz and 16 GB

⁵[Online]. Available: <https://itea3.org/project/a2nets.html>

⁶[Online]. Available: <https://www.enOcean.com/en/home/>

⁷[Online]. Available: <http://www.bacnet.org/>

random memory. Without loss of generality, composite services considered in the simulation scenarios have a sequential structure. Other structures can be transformed into sequential structures using existing techniques [50]. The scenarios are generated by varying the number of services classes m , and the number of concrete services per class n . For each concrete service, three QoS attributes are evaluated: cost, availability, and reliability.

Due to the complexity of establishing a large-scale IoT deployment and due to the absence in the literature of datasets including both QoS and energy profile values of IoT services, the simulation data were synthetically generated: 1) according to the values reported in the literature (e.g., [14], [19]) for the QoS attributes, and 2) using the realistic energy model (obtained from a measurement data) described in [51] to specify the energy profile of IoT services. The availability and reliability are generated assuming a uniform distribution over the interval [0.95, 0.99999]. The cost of services is generated according to a uniform distribution over the interval [10, 20]. Fluctuations of the QoS values are considered as follows: at iteration $t+1$ of the selection process, the QoS value $qos_{q,j}^i(t+1)$ of each attribute is randomly chosen in the interval $[0.9qos_{q,j}^i(t), 1.1qos_{q,j}^i(t)]$ where $qos_{q,j}^i(t)$ represents the value of this attribute at time t . Finally, the importance or weight of each QoS attribute is randomly generated under the condition that the sum of all the weight values is equal to 1. In the energy model described in [51], the battery of each device has an initial charge value $C_{initial}$, chosen randomly in the interval $[0.7 C_{max}, 1.0 C_{max}]$, where C_{max} represents the maximum battery charge. This model has the advantage to consider services with different autonomies. Each invocation of a concrete service induces an average energy consumption. When a service is requested, a charge chosen randomly in the interval [100 mA.s, 10 000 mA.s] is subtracted from the actual battery charge of the device hosting the service. A device stops providing a service when a critical battery level $C_{Threshold}$ is reached. This service is then considered failed from the selection point of view. The maximum battery capacity of a device is $C_{max} = 1500 \text{ mA} \cdot \text{h}$, whereas the critical battery level $C_{Threshold}$ is set to 30% of the maximum battery charge.

In order to show the added value of the proposed selection approach in a large-scale IoT services environment, three algorithms are evaluated.

- *EQSA*: where the energy profile and QoS are taken into account in the selection process.
- *EQSA ~ EP*: a variant of *EQSA* where only the energy profile is taken into account in the selection process, i.e., the services having the minimum energy profile are selected.
- *EQSA ~ QoS*: a variant of *EQSA* where only the QoS is taken into account in the selection process, i.e., the services offering the best QoS-based dominance are selected.

The results presented hereafter correspond to the average of 100 simulations.

B. Simulation Results

The performances of the aforementioned algorithms are evaluated through the following metrics.

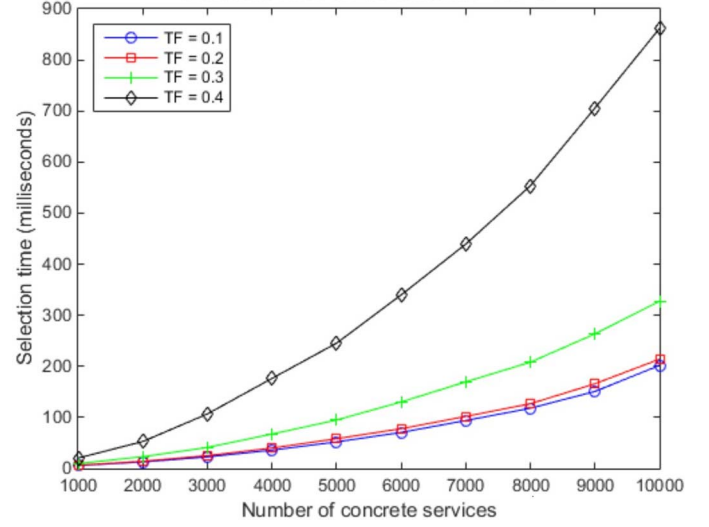


Fig. 1. Selection time versus Number of concrete services (10 services classes).

- *Selection time*: it represents the computational time of the selection algorithm.
- *Relative dominance* of the composite service: it is equal to the sum of the relative dominances of the atomic services composing the composite service.
- *QoS-based dominance* of the composite service: it is equal to the sum of the QoS-based dominances of the atomic services forming the composite service.
- *Energy consumption* of the composite service: it is equal to the sum of energies consumed by its components.
- *Composition lifetime*: it is defined as the number of compositions that can be executed before the first concrete service failure. A service is considered failed when its autonomy is no longer sufficient to be invoked.
- *Optimality*: this metric is defined as the ratio between the quality value of the composite service obtained in the case of the *EQSA* algorithm and the optimal quality value of the composite service, obtained in the case of the *EQSA ~ QoS* algorithm. Formally

$$Optimality = \frac{\sum_{q=1}^{|QoS|} Optimality(qos_q)}{|QoS|}. \quad (17)$$

with:

$$Optimality(qos_q) = \begin{cases} \frac{\frac{1}{qos_q(EQSA)}}{\frac{1}{qos_q(EQSA \sim QoS)}}, & \text{if } q \text{ is negative} \\ \frac{qos_q(EQSA)}{qos_q(EQSA \sim QoS)}, & \text{otherwise} \end{cases}. \quad (18)$$

where qos_q is the q^{th} QoS attribute and $|QoS|$ is the number of QoS attributes. $qos_q(EQSA)$ and $qos_q(EQSA \sim QoS)$ are the values of the q^{th} QoS attribute of the composite service obtained using *EQSA* and *EQSA ~ QoS* algorithms, respectively.

1) *Selection Time Versus Number of Services*: The first simulation is intended to validate the scalability of the *EQSA* algorithm. The number of services classes is set to 10, which correspond to a typical structure of a composition plan in the ad-

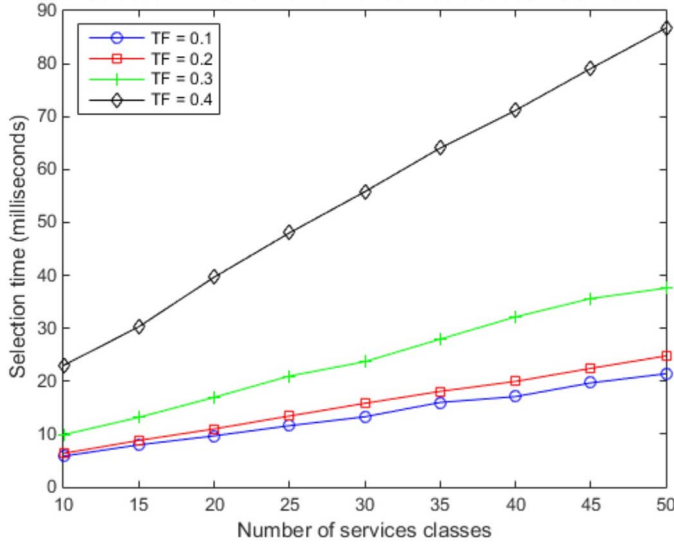


Fig. 2. Selection time versus number of services classes (1000 concrete services per class).

addressed use-case. The number of candidate services for each class varies from 1000 to 10000. As shown in Fig. 1, the selection time increases with the tolerance factor as the number of candidate services satisfying the QoS threshold values increases. The average selection time taken by the *EQSA* algorithm is compatible with large scale IoT environments since it does not exceed 900 milliseconds for a number of concrete services equal to 10 000.

In the second simulation, the number of services classes varies from 10 to 50 classes whereas the number of concrete services is set to 1000 for each class. The bound of 50 services classes corresponds to an extreme but realistic case of services composition. As observed in Fig. 2, the selection time increases with the number of services classes used in the composition structure. Similarly to the first simulation, the selection time increases with the tolerance factor. In addition, when this factor becomes larger ($TF = 0.4$), the number of preselected candidate services increases significantly as well as the selection time that does not exceed 90 ms for a number of services classes equal to 50.

2) *Relative Dominance Versus Number of Services*: In the third simulation, the performances of *EQSA*, *EQSA ~ EP* and *EQSA ~ QoS* algorithms are evaluated in terms of *relative dominance*. The number of concrete services for each class varies from 100 to 1000 services whereas the number of services classes and the tolerance factor are set respectively to 10 classes and 0.3. It should be noted that the value 0.3 corresponds to the maximum degree of tolerance beyond which the user's requirements are considered as not satisfied. As shown in Fig. 3, the relative dominance of the three selection algorithms increases with the number of concrete services. This is due to the increase of the EP-based and QoS-based dominances. Fig. 3 shows also that the relative dominance in the case of the *EQSA* algorithm is better than that obtained in the case of *EQSA ~ EP* and *EQSA ~ QoS* algorithms. This can be explained by the fact that the *EQSA* algorithm allows selecting candidate services

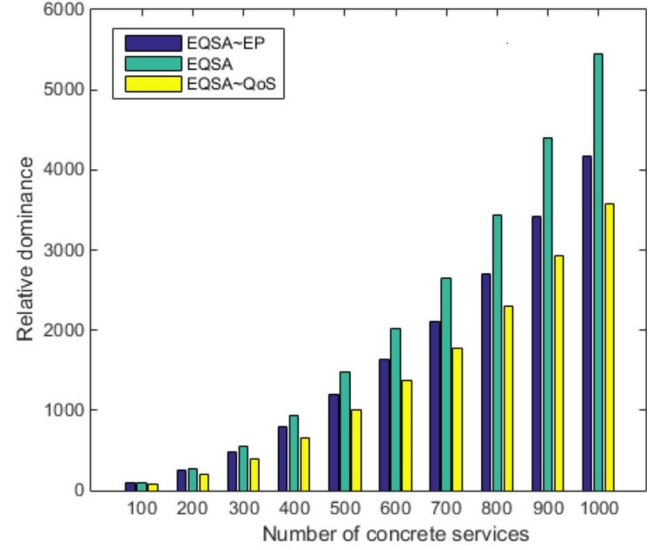


Fig. 3. Relative dominance versus number of concrete services.

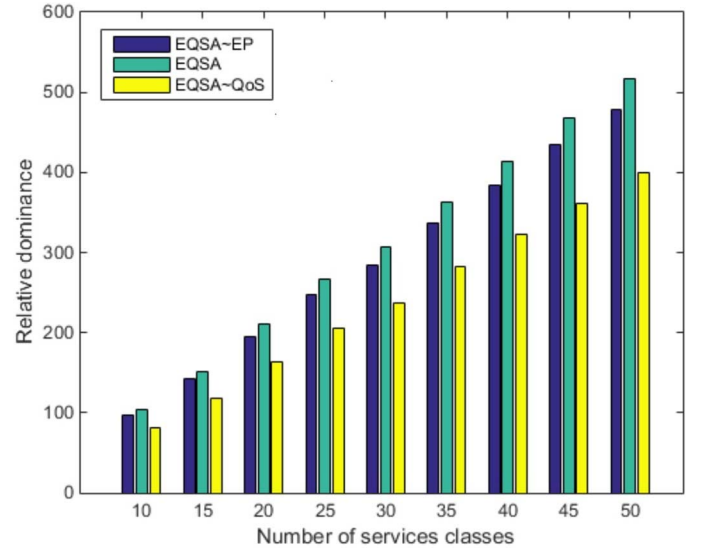


Fig. 4. Relative dominance versus number of services classes.

maximizing both EP-based dominance and QoS-based dominance. It can be noticed that *EQSA ~ QoS* (*EQSA ~ EP* respectively) algorithm selects the services regardless of their EP-based (QoS-based respectively) dominance. Services with low EP-based (QoS-based respectively) dominance are likely to be selected, leading therefore to a reduction of the relative dominance of the composite service.

In the fourth simulation, we have measured the *relative dominance* by varying the number of services classes from 10 to 50 classes. The number of candidate services for each class and the tolerance factor are set to 100 and 0.3, respectively. Fig. 4 shows that the relative dominance of the three algorithms increases linearly with the number of abstract services. Indeed, when the size of the composition structure becomes high, the number of selected services increases.

3) *QoS-Based Dominance Versus Number of Services*: In the fifth simulation, the performances of *EQSA*, *EQSA ~ EP* and

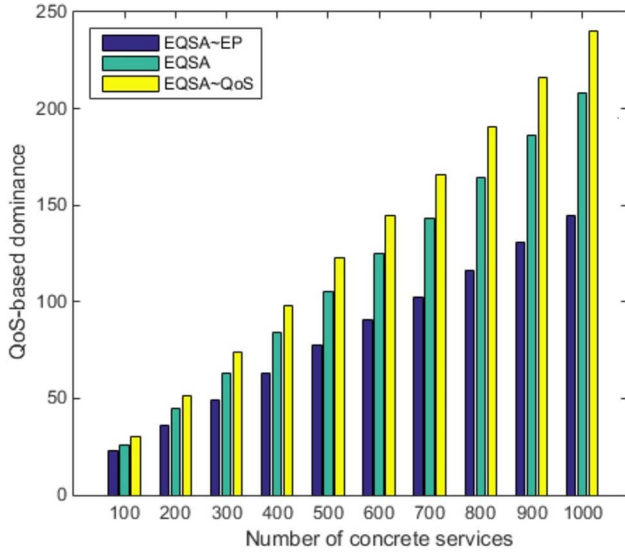


Fig. 5. QoS-based dominance versus number of concrete services.

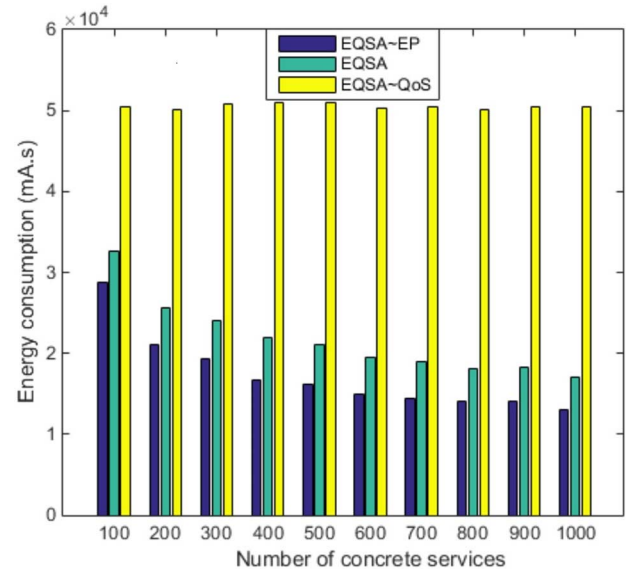


Fig. 7. Energy consumption versus number of concrete services.

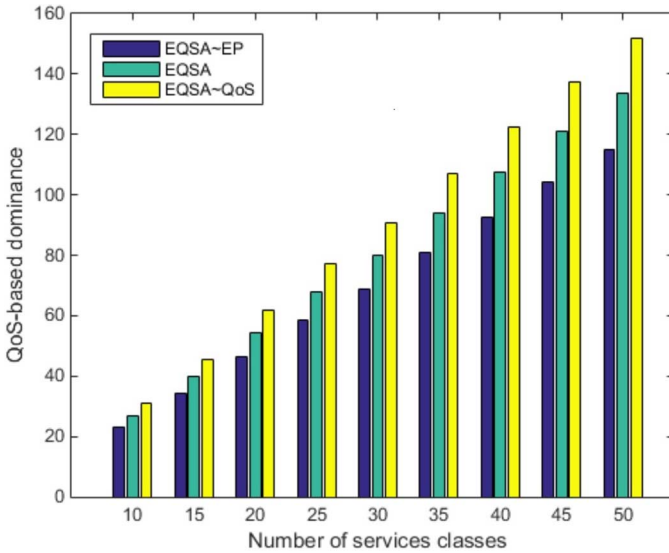


Fig. 6. QoS-based dominance versus number of services classes.

EQSA ~ QoS algorithms are compared in terms of *QoS-based dominance*. The number of services classes and the tolerance factor are set to 10 and 0.3 respectively, and the number of concrete services per class varies from 100 to 1000. As shown in Fig. 5, the QoS-based dominance obtained with the EQSA algorithm is slightly lower than that obtained with the EQSA ~ QoS algorithm. This is due to the principle of the EQSA algorithm that consists of decreasing slightly the QoS to make energy conservation. The QoS-based dominance ratio between the EQSA algorithm and the EQSA ~ QoS algorithm is stable; it varies from 87.5% for the lowest number of candidate services (100 services) to 86.65% for the highest one (1000 services). The QoS of the resulting composite service is not affected when the number of concrete services increases.

In the sixth simulation, the *QoS-based dominances* obtained with EQSA, EQSA ~ EP and EQSA ~ QoS algorithms are evaluated by varying the number of services classes from 10 to

50. The number of concrete services in each class and the tolerance factor are set to 100 and 0.3, respectively. Fig. 6 shows that the QoS-based dominance increases with the number of services classes. Indeed, as explained in the fourth scenario, the number of selected services increases with the number of services forming a composite service.

4) *Energy Consumption Versus Number of Services*: In the seventh simulation, a composition plan of 10 service classes is considered. For each class, the number of candidate services varies from 100 to 1000. The tolerance factor is set to 0.3. The performances of the three algorithms are compared in terms of *energy consumption* measured in mA.s as defined in Section VII-A. Fig. 7 shows that the energy consumption using the EQSA algorithm is close to that obtained with the EQSA ~ EP algorithm where only services having the lowest energy profile are selected. In terms of energy, the EQSA algorithm consumes from 13% to 31% more energy than the EQSA ~ EP algorithm when the number of services varies between 100 and 1000. Indeed, the EQSA algorithm allows obtaining a composite service with energy consumption close to that obtained in the case of the EQSA ~ EP algorithm. The latter provides the best composite service in terms of minimal energy consumption. Fig. 7 shows also that when the number of concrete services increases, the energy consumption decreases in the case of the EQSA algorithm. This is explained by the fact that when the number of candidate services increases, the probability of finding more services consuming less energy increases in the same manner. Furthermore, another positive impact of the EQSA algorithm, is that it does not reduce significantly the QoS of the composite service while conserving between 36% and 67% of the energy in comparison to the EQSA ~ QoS algorithm when the number of services varies between 100 and 1000.

In the eighth simulation, the performances of EQSA, EQSA ~ EP and EQSA ~ QoS are compared in terms of *energy consumption* of the composite service. The number of services classes varies from 10 to 50 whereas the number of

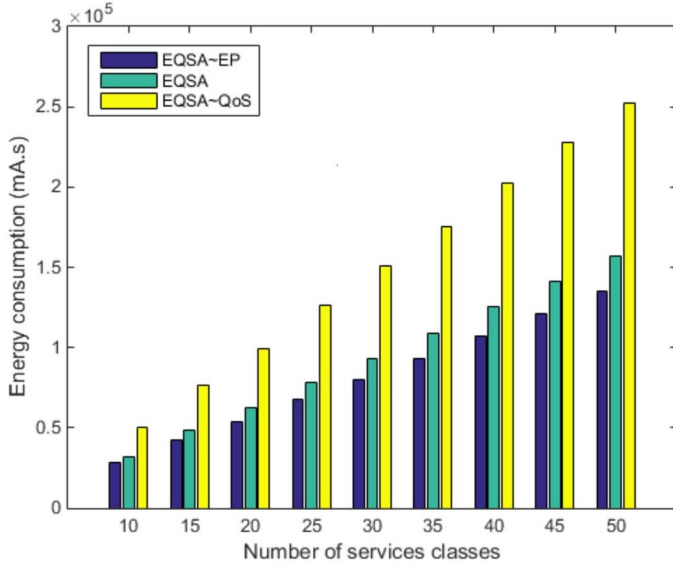


Fig. 8. Energy consumption versus number of services classes.

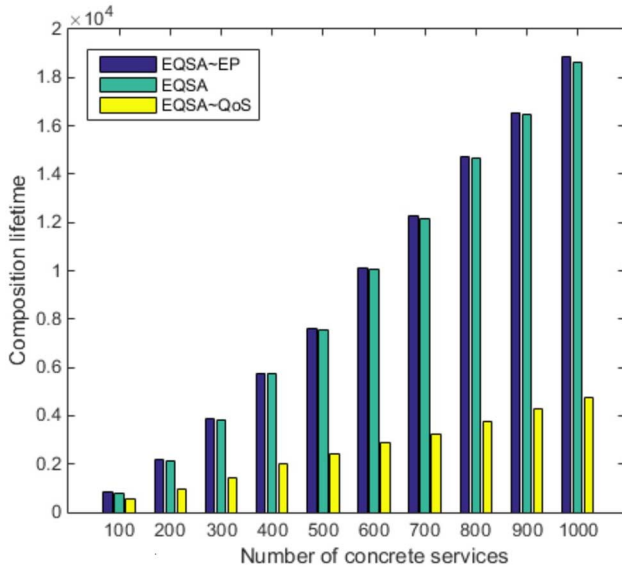


Fig. 9. Composition lifetime versus number of concrete services.

concrete services in each class and the tolerance factor are set to 100, and 0.3, respectively. Fig. 8 shows that the energy consumption of the composite service increases due to the fact that the number of selected services increases linearly with the size of the composition plan.

5) *Composition Lifetime Versus Number of Concrete Services*: The aim of the ninth simulation is to assess and compare the *composition lifetime* of *EQSA*, *EQSA ~ EP* and *EQSA ~ QoS* algorithms. The composition plan is composed of 10 services classes. For each service class, the number of concrete services varies from 100 to 1000, and the tolerance factor is set to 0.3. Fig. 9 shows that the *EQSA* algorithm is slightly less efficient in terms of composition lifetime compared to the *EQSA ~ EP* algorithm. The latter provides an optimal composition lifetime while reducing the QoS of the composite service. The proposed approach ensures, in terms of selection,

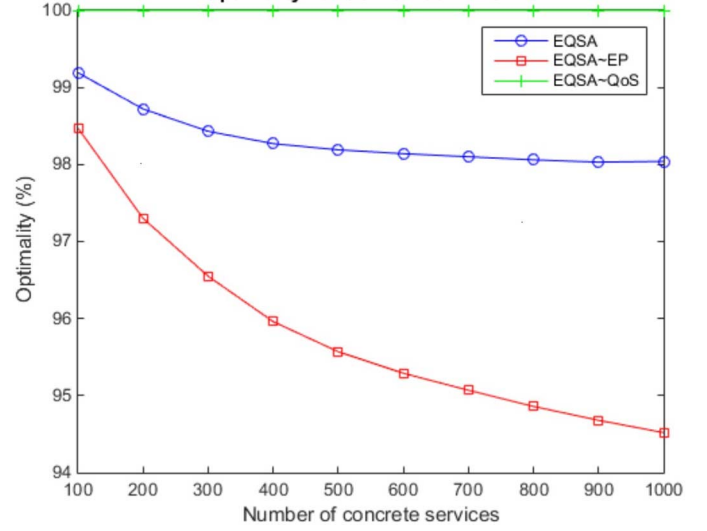


Fig. 10. Optimality QoS versus number of concrete services.

a good balance among the candidate services by invoking the services consuming less energy. This balance ensures a high availability of services. Indeed, the composition lifetime in the case of the *EQSA* algorithm is close to that obtained with the *EQSA ~ EP* algorithm by consuming a reasonable additional amount of energy (about 13 to 31%) as shown in Fig. 7.

6) *Optimality of the Solution*: In the last simulation, *EQSA*, *EQSA ~ EP* and *EQSA ~ QoS* algorithms are compared in terms of *optimality* of solutions. The composite service consists of 10 services classes. The number of concrete services varies from 100 to 1000 for each class, and the tolerance factor is set to 0.3. Fig. 10 shows that the optimality of the proposed approach decreases slightly when the number of candidate services increases, but remains stable beyond 700 services. Unlike the *EQSA ~ EP* algorithm whose optimality decreases significantly and in continuous manner, the *EQSA* algorithm is able to find very close-to-optimal solutions (about 98%) even in the case of large scale services environments.

VIII. CONCLUSION AND PERSPECTIVES

In this paper, an energy-centered and QoS-aware services selection approach for IoT environments is presented. Formulated and solved as a multi-objective optimization problem, this approach allows minimizing energy consumption to ensure a high availability of composite services while satisfying the user's QoS requirements. A QoS level-based services pre-selection is first achieved using a lexicographic optimization method where each sub-problem deals with one QoS attribute. By introducing the concept of relative dominance relation in the sense of Pareto, the preselected candidate services are then compared to select the best service. The relative dominance of a service is calculated from its energy profile and QoS attributes, and user's preferences. Simulation results highlight the benefit and the promising performances of the proposed approach. The *EQSA* algorithm is scalable in time performance for large-scale IoT environments and is able to find very close-to-optimal solutions (about 98%). In addition, it gives a composition lifetime close to the optimal case by consuming a reasonable

additional amount of energy (about 13 to 31%). In terms of perspectives of this research work, an interesting topic to address concerns the extension of the EQSA algorithm to handle the dynamic re-selection of services by predicting QoS and Energy parameters changes, with respect to the different composition operators used in a given plan. Another topic to study is the extension of the proposed approach to handle the automatic generation of an energy efficient services composition plan that can evolve with the IoT environment and users' expectations in terms of QoS. Such extensions are of particular importance to deal with situations where no selection solution can be found or when IoT services fail during run-time.

REFERENCES

- [1] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A survey on enabling technologies, protocols and applications," *IEEE Commun. Surveys Tuts.*, vol. 17, no. 4, pp. 2347–2376, 2015.
- [2] D. Giusto, A. Iera, G. Morabito, and L. Atzori, *The Internet of Things: 20th Tyrrhenian Workshop on Digital Communications*. New York, NY, USA: Springer Science & Business Media, 2010.
- [3] L. Li, S. Li, and S. Zhao, "QoS-aware scheduling of services-oriented internet of things," *IEEE Trans. Ind. Inf.*, vol. 10, no. 2, pp. 1497–1505, May 2014.
- [4] C. Jatoth, G. Gangadharan, and R. Buyya, "Computational intelligence based QoS-aware web service composition: A systematic literature review," *IEEE Trans. Services Computing*, vol. PP, no. 99, 2015, DOI: 10.1109/TSC.2015.2473840.
- [5] D. Guinard, V. Trifa, S. Karnouskos, P. Spiess, and D. Savio, "Interacting with the SOA-based internet of things: Discovery, query, selection, and on-demand provisioning of web services," *IEEE Trans. Services Computing*, vol. 3, no. 3, pp. 223–235, Jul.–Sep. 2010.
- [6] S. Robinson, "Cellphone Energy Gap: Desperately Seeking Solutions," Strategy Analytics, Newton, MA, USA, Tech. Rep., 2009.
- [7] A. Yachir, Y. Amirat, A. Chibani, and N. Badache, "Event-aware framework for dynamic services discovery and selection in the context of ambient intelligence and Internet of Things," *IEEE Trans. Autom. Sci. Eng.*, vol. 13, no. 1, pp. 85–102, Jan. 2015.
- [8] A. Yachir, K. Tari, A. Chibani, and Y. Amirat, "Towards an automatic approach for ubiquitous robotic services composition," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots and Syst.*, Nice, France, 2008, pp. 3717–3724.
- [9] A. Yachir, K. Tari, Y. Amirat, A. Chibani, and N. Badache, "QoS based framework for ubiquitous robotic services composition," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, St. Louis, MO, USA, 2009, pp. 2019–2026.
- [10] A. Kouicem, A. Chibani, A. Tari, Y. Amirat, and Z. Tari, "Dynamic services selection approach for the composition of complex services in the web of objects," in *Proc. IEEE World Forum Internet of Things*, Seoul, Korea, 2014, pp. 298–303.
- [11] A. Yachir, Y. Amirat, A. Chibani, and N. Badache, "Towards an event-aware approach for ubiquitous computing based on automatic service composition and selection," *Ann. Telecommun.*, vol. 67, no. 7–8, pp. 341–353, Jun. 2012.
- [12] L. Zeng, B. Benatallah, A. H. Ngu, M. Dumas, J. Kalagnanam, and H. Chang, "Qos-aware middleware for Web services composition," *IEEE Trans. Softw. Eng.*, vol. 30, no. 5, pp. 311–327, May 2004.
- [13] Y. Ngoko, A. Goldman, and D. Milojicic, "Service selection in web service compositions optimizing energy consumption and service response time," *J. Internet Services and Applicat.*, vol. 4, no. 1, pp. 1–12, Dec. 2013.
- [14] D. Ardagna and B. Pernici, "Adaptive service composition in flexible processes," *IEEE Trans. Softw. Eng.*, vol. 33, no. 6, pp. 369–384, Jun. 2007.
- [15] T. Yu, Y. Zhang, and K.-J. Lin, "Efficient algorithms for Web services selection with end-to-end QoS constraints," *ACM Trans. Web*, vol. 1, no. 1, pp. 6:1–6:26, May 2007.
- [16] G. A. Garcia Llinas and R. Nagi, "Network and QoS-based selection of complementary services," *IEEE Trans. Services Computing*, vol. 8, no. 1, pp. 79–91, Feb. 2015.
- [17] Q. Wu and Q. Zhu, "Transactional and QoS-aware dynamic service composition based on ant colony optimization," *Future Generation Comput. Syst.*, vol. 29, no. 5, pp. 1112–1119, Jul. 2013.
- [18] M. Alrifai, T. Risse, and W. Nejdl, "A hybrid approach for efficient Web service composition with end-to-end QoS constraints," *ACM Trans. Web*, vol. 6, no. 2, pp. 7:1–7:31, May 2012.
- [19] S. X. Sun and J. Zhao, "A decomposition-based approach for service composition with global QoS guarantees," *Inf. Sci.*, vol. 199, pp. 138–153, Sep. 2012.
- [20] F. Mardukhi, N. Nematbakhsh, K. Zamanifar, and A. Barati, "QoS decomposition for service composition using genetic algorithm," *Appl. Softw. Computing*, vol. 13, no. 7, pp. 3409–3421, Jul. 2013.
- [21] M. Zeleny and J. L. Cochrane, *Multiple Criteria Decision Making*. Columbia, SC, USA: Univ. South Carolina, 1973.
- [22] N. B. Mabrouk, S. Beauche, E. Kuznetsova, N. Georgantas, and V. Issarny, "QoS-aware service composition in dynamic service oriented environments," in *Proc. ACM/IFIP/USENIX, 10th Int. Middleware Conf.*, USA, 2009, pp. 123–142.
- [23] S. Hwang, C.-T. Hsu, and C.-K. Lee, "Service selection for web services with probabilistic QoS," *IEEE Trans. Services Computing*, vol. 8, no. 3, pp. 467–480, May–Jun. 2015.
- [24] Y. Chen, J. Huang, C. Lin, and J. Hu, "A partial selection methodology for efficient QoS-aware service composition," *IEEE Trans. Services Computing*, vol. 8, no. 3, pp. 384–397, May–Jun. 2015.
- [25] H. Al-Helal and R. Gamble, "Introducing replaceability into web service composition," *IEEE Trans. Services Computing*, vol. 7, no. 2, pp. 198–209, Apr.–Jun. 2014.
- [26] I. Trummer, B. Faltings, and W. Binder, "Multi-objective quality-driven service selection—A fully polynomial time approximation scheme," *IEEE Trans. Softw. Eng.*, vol. 40, no. 2, pp. 167–191, Feb. 2014.
- [27] X. Jin, S. Chun, J. Jung, and K.-H. Lee, "IoT service selection based on physical service model and absolute dominance relationship," in *Proc. 7th IEEE Int. Conf. Service-Oriented Computing and Applicat.*, Matsue, Japan, 2014, pp. 65–72.
- [28] C. Vira and Y. Y. Haimes, *Multiobjective Decision Making: Theory and Methodology*. Amsterdam, The Netherlands: North-Holland, 1983.
- [29] E. Zitzler, L. Thiele, M. Laumanns, C. M. Fonseca, and V. G. D. Fonseca, "Performance assessment of multiobjective optimizers: An analysis and review," *IEEE Trans. Evol. Computation*, vol. 7, no. 2, pp. 117–132, Apr. 2003.
- [30] A. M. Ferreira, K. Kritikos, and B. Pernici, "Energy-aware design of service-based applications," in *Proc. 7th Int. Joint Conf. Service-Oriented Computing*, Stockholm, Sweden, 2009, pp. 99–114.
- [31] A. Beloglazov, J. Abawajy, and R. Buyya, "Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing," *Future Generation Comput. Syst.*, vol. 28, no. 5, pp. 755–768, May 2012.
- [32] F. Tao, Y. LaiLi, L. Xu, and L. Zhang, "FC-PACO-RM: A parallel method for service composition optimal-selection in cloud manufacturing system," *IEEE Trans. Ind. Inf.*, vol. 9, no. 4, pp. 2023–2033, Nov. 2013.
- [33] A. Jula, E. Sundararajan, and Z. Othman, "Cloud computing service composition: A systematic literature review," *Expert Syst. Applicat.*, vol. 41, no. 8, pp. 3809–3824, Jun. 2014.
- [34] F. Xiang, Y. Hu, Y. Yu, and H. Wu, "Qos and energy consumption aware service composition and optimal-selection based on pareto group leader algorithm in cloud manufacturing system," *Central Eur. J. Oper. Res.*, vol. 22, no. 4, pp. 663–685, Dec. 2014.
- [35] G. Tanganelli, C. Vallati, and E. Mingozzi, "Energy-efficient QoS-aware service allocation for the cloud of things," in *Proc. 6th IEEE Int. Conf. Cloud Computing Technol. and Sci.*, Singapore, 2014, pp. 787–792.
- [36] J. Pan, R. Jain, S. Paul, T. Vu, A. Saifullah, and M. Sha, "A internet of things framework for smart energy in buildings: Designs, prototype, and experiments," *IEEE Internet of Things J.*, vol. 2, no. 6, pp. 527–537, Dec. 2015.
- [37] A. D. Paola, M. Ortolani, G. L. Re, G. Anastasi, and S. K. Das, "Intelligent management systems for energy efficiency in buildings: A survey," *ACM Computing Surveys*, vol. 47, no. 1, pp. 13:1–13:38, Jul. 2014.
- [38] S. Ehsan and B. Hamdaoui, "A survey on energy-efficient routing techniques with QoS assurances for wireless multimedia sensor networks," *IEEE Commun. Surveys & Tutorials*, vol. 14, no. 2, pp. 265–278, 2012.
- [39] Y. Ngoko, C. Cérin, and A. Goldman, "A multithreaded resolution of the service selection problem based on domain decomposition and work stealing," in *Proc. IEEE Int. Conf. Services Computing*, Anchorage, AK, USA, 2014, pp. 424–431.
- [40] D. Geebelen *et al.*, "QoS prediction for web service compositions using kernel-based quantile estimation with online adaptation of the constant offset," *Inf. Sci.*, vol. 268, pp. 397–424, Jun. 2014.

- [41] Y. Wu *et al.*, "A novel method for calculating service reputation," *IEEE Trans. Autom. Sci. Eng.*, vol. 10, no. 3, pp. 634–642, Jul. 2013.
- [42] P. Xiong, Y. Fan, and M. Zhou, "Web service configuration under multiple quality-of-service attributes," *IEEE Trans. Autom. Sci. Eng.*, vol. 6, no. 2, pp. 311–321, Apr. 2009.
- [43] E.-G. Talbi, *Metaheuristics: From Design to Implementation*. Hoboken, NJ, USA: Wiley, 2009.
- [44] P. C. Fishburn, "Exceptional paper-lexicographic orders, utilities and decision rules: A survey," *Manag. Sci.*, vol. 20, no. 11, pp. 1442–1471, Jul. 1974.
- [45] K. Deb, A. Pratap, and S. A. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Computation*, vol. 6, no. 2, pp. 182–197, Apr. 2002.
- [46] G. Mulligan, "The 6LoWPAN architecture," in *Proc. 4th Workshop on Embedded Networked Sensors*, 2007, pp. 78–82.
- [47] C. Gomez and J. Paradells, "Wireless home automation networks: A survey of architectures and technologies," *IEEE Commun. Mag.*, vol. 48, no. 6, pp. 92–101, Jun. 2010.
- [48] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.
- [49] C. Bormann, A. P. Castellani, and Z. Shelby, "COAP: An application protocol for billions of tiny internet nodes," *IEEE Internet Computing*, vol. 16, no. 2, pp. 62–67, Mar.–Apr. 2012.
- [50] J. Cardoso, A. Sheth, J. Miller, J. Arnold, and K. Kochut, "Quality of service for workflows and web service processes," *Web Semantics: Sci., Services and Agents on World Wide Web*, vol. 1, no. 3, pp. 281–308, Apr. 2004.
- [51] J. Furthmüller and O. P. Waldhorst, "Energy-aware resource sharing with mobile devices," *Comput. Networks*, vol. 56, no. 7, pp. 1920–1934, May 2012.



Mohamed Essaid Khanouche received the Engineering degree and Magister degree (first level of the Post-graduation) in computer science from University of Bejaia, Bejaia, Algeria, in 2005 and 2010, respectively, where he is currently working toward the Ph.D. degree at the Laboratory of Medical Computing (LIMED) and Laboratory of Signal and Intelligent Systems (LISSI) – University of Paris Est-Créteil, France.

His research interests are ubiquitous computing, Internet of Things, service-oriented computing, energy efficiency, QoS management and optimization of services.



Yacine Amirat (M'13) received the Ph.D. degree in computer science and robotics from the University of Pierre et Marie Curie, Paris, France, in 1989, and the Habilitation degree in artificial intelligence and control of complex systems from the University of Paris 12-France, Paris, in 1996.

He cocreated the Laboratory of Computer Sciences and Robotics, Paris 12 University (newly UPEC), Créteil, France, in 1990. He is currently the Director of the Laboratory of Image, Signal and Intelligent Systems (LISSI)—UPEC, and the Scientific Director of several research projects. He has authored or coauthored more than 100 papers in scientific journals, books, and conference proceedings. His current research interests include artificial intelligence, soft computing, knowledge processing, and control of complex systems. Application fields are

intelligent and robotic systems such as ubiquitous and service robots, wearable robots, and ambient intelligence.

Prof. Amirat is a reviewer of many international journals and conferences and a Technical Committee Member, the Session Chair, a Session Organizer, or an Associate Editor of several IEEE-RAS conferences. He is actively collaborating with industrial and academic partners in various European projects.



Abdelghani Chibani received the M.Sc. degree in computer science from Paris 6 University, Paris, France, in 2001, and the Ph.D. degree in computer science from Paris Est Créteil (UPEC), Créteil, France, in 2006.

After more than nine years as a Senior Consultant in the computer industry, in 2008 he joined the UPEC University as an Associate Professor. He is managing collaborative research projects of the SIRIUS team at the LISSI lab that are funded by different European and national programs. He served as work package manager in the several European projects: ITEA2 MULTIPOL 2008–2010, ICT PSP PEOPLE 2011–2013, ITEA2 PREDYKOT 2012–2014 and ITEA2 Web-of-Objects 2012–2015. He is an active member of the IEEE RAS working group for the standardization of ontologies for Robotics and Automation (ORA). His research interests are: context awareness, activities and situations recognition, service composition techniques, QoS policies, ontologies, and commonsense reasoning. Application domains are: ambient intelligence and ubiquitous–cloud robotics.



Moussa Kerkar received the M.Sc. and Ph.D. degrees from the University of Warwick, Warwick, U.K., in 1986 and 1989, respectively.

He then spent two years as a Post-Doctoral Research Fellow in surface science with the Physics Department of the Faculty of Science, University of Warwick, U.K. He was appointed a Lecturer and then a Senior Lecturer with Ecole Nationale Polytechnique d'Alger (ENP, 1992–1997) and moved to the University Abderrahmane Mira of Béjaia, Algeria, where he has been a Full Professor in physics since 2000. He is head of a Research Group of the L2ME laboratory and, at the present moment, he is spending one sabbatical year since September 2015 at LISSI (UPEC). He is involved in different international research projects, collaborating mainly with different European laboratories. He has also occupied various high positions such as Vice-Rector of the University of Béjaia and Dean of the Faculty of Exact Sciences of the University of Béjaia.



Ali Yachir received the M.Sc. degree in computer science from Béjaia University, Béjaia, Algeria, in 2008, and the Ph.D. from USTHB University, Algeria, and UPEC University, Créteil, France, in 2014.

He is a Researcher in wireless networks and ambient intelligent systems with the Polytechnic School of Algiers (EMP). His current research interests include wireless sensor networks, ambient intelligent systems, Internet and Web of things and service-oriented architectures. Application fields are services discovery, selection, composition and monitoring in the context of ambient assisted living (AAL) domains.